PERSONAL PROJECT

AI News Shorts Automation

Automating news collection, selection, scripting, visual generation, narration and editing

Period: Jan - Mar 2026
Role: Planning, development and channel operations

01 Collect Articles
Puppeteer / Sheets

→

02 Select Articles
GPT scoring / code-based ranking

→

03 Types & Plots
Tags / six types / slots

→

04 Scripts & Scenes
Slot scripts / scene JSON

Visuals
Naver image search → Claude review → Gemini images → Veo videos

Voice & Editing
Slot scripts → Naver TTS → Whisper timestamps → subtitles / FFmpeg

Shared keys: row_number (article) + slot index (scene) / JSON files and assets connect each stage

PROJECT OVERVIEW

Connecting AI outputs to the next task

Stage-specific JSON files and identifiers connect collection, selection, scripting, scenes and media generation.

Collection & Selection

Puppeteer / Google Sheets

GPT headline and article scoring

Scripts & Scenes

Tags → type-specific plots → slot scripts

Scene JSON: people, settings, media types

Media Production

Naver search / Claude review and selection

Gemini / Veo / voice / subtitle editing

How I connected the stages

I used row_number to retain article identity and slot indices to link scripts with scene files. JSON files and directories store intermediate outputs, making each stage's inputs and outputs available for inspection.

A common format for multiple news sources

Collect headlines and URLs first, then retrieve the full text of shortlisted articles.

01 Headline Collection

Call source-specific scrapers for MBC, JTBC and Hankyoreh.
Each source has its own try/catch block, so one failure
does not prevent collection from the remaining sources.

02 Full-Text Extraction

Select the extraction function using the media field.
Normalize sentence breaks, then add the article body
and its length to the existing article object.

detail_index.js

```
const formattedText = formatArticleText(fullText);
enrichedArticles.push({
  ...article,
  본문: formattedText,
  본문길이: formattedText.length,
});
```

Attach full text to the original article

Spread the article object to retain its URL and ID.
Store each source's text in the same fields
so the scoring stage can read a consistent format.

Source: index.js, detail_index.js, googleSheet.js, get_googlesheet.js. Source-specific parsers are separate modules.

Model scoring with code-controlled selection

A two-stage process narrows candidates by headline, then evaluates the full article.

Headline Scoring

GPT-4.1-mini scores four criteria

Promise.all runs batches of 10

Code sums the overall_score

Filtering & Shortlisting

Map removes identical headlines

Removal reasons saved as JSON

Shortlist 2× the target video count

Article Scoring & Selection

Score five full-text criteria

Generate HOOK_1 / HOOK_2

Select NUM_OF_VIDEOS articles

```
scores.overall_score =
  scores.clickbait_score + scores.emotional_intensity +
  scores.exaggeration_level + scores.information_gap;
scores.row_number = rowNumber;
```

GPT returns scores for each criterion

Code sets the total and article identifier

row_number links back to the source article

Source: sort_title.js, sort_title_scoring.js, sort_title_duplicate.js, sort_content.js, sort_15.js. Channel-specific filtering runs separately.

Predefined plots for six article types

Priority-ordered conditions map core_tag and secondary_tags to an article_class.

Article Type	Classification Conditions	Predefined Slot Sequence
TIMING_FOCUS	DELAY tag takes highest priority	HOOK → SUMMARY → DELAY → MOTIVE → RESPONSIBILITY → TIMING_Q
CONFLICT_FOCUS	REACTION / CONFLICT / POWER_STRUGGLE	HOOK → A → B → CLASH → ESCALATION → JUDGMENT → POLL_Q
CONTROVERSY_FOCUS	SCANDAL / MEDIA_ISSUE / PUBLIC_OPINION	HOOK → ISSUE → FACT → RESPONSE → DOUBT → SUSPENSE_Q
RESPONSIBILITY_FOCUS	EXPLANATION / HISTORICAL_REFERENCE	HOOK → CONTEXT → EXPLANATION → STANDARD → JUDGMENT → RESPONSIBILITY_Q
PROCESS_FOCUS	core_tag: LEGAL / INVESTIGATION / RULING	HOOK → STATUS → PROCEDURE → PROBLEM → NEXT → LEGITIMACY_Q
FACT_FOCUS	Default when no earlier rule matches	HOOK → SETUP → DELAY → SUSPENSE_Q → CTA → ANSWER

Code defines the type and slot order, then passes each slot to the script generation stage.

Source: classifyArticleClass and structureMap in get_slot.js. Conditions are evaluated top-down; tag scope follows the code.

Roles and writing rules for every script slot

A slot is one segment of the video. Its order, role and guidelines are defined before script generation.

Conflict-focused example

HOOK → A → B → CLASH → ESCALATION → JUDGMENT → POLL_Q

```
B: {
  role: "Summarize the opposing side with equal weight.",
  guidelines: "Match A in tone and length.",
  intent: "Establish the competing positions"
}

// Add role / guidelines / intent to each slot
```

role: what this segment should explain

guidelines: rules for writing the segment

intent: the purpose of the segment

Pass the article body alongside the slot rules to generate a script for the selected plot.

tagged_articles.json → slotted_articles.json → processScripts(...) → scripted_articles.json

Source: slotRoleMap and processSlots in get_slot.js / write_slot.js. These rules guide generation; they do not automatically validate compliance.

Scene JSON connects image search and generation

scene_describer specifies the people, setting and scene description for each slot.

```
{
  "row_number": ${article.row_number},
  "slots": [
    {
      "index": number,
      "media_type": "image" | "video",
      "주인공 수": "1" | "2",
      "주인공1": "Real name in Korean",
      "주인공2": "Real name in Korean or null",
      "배경": "Real location in Korean",
      "장면 설명": "Describe protagonist1 / protagonist2"
    }
  ]
}
```

How the fields are used

row_number / index

Link each article to its scene files

Person 1 / Person 2 / Background

Identify subjects for image search

media_type / Scene description

Inputs for image or video generation

Prompt rules

Preserve slot count and media_type

Refer to people as protagonist1 / protagonist2

Prompt schema with types and placeholders. Original Korean keys retained. Source: scene_describer.js / scene_describe.js.

Naver image search with Claude reference review

Use people and settings from scene JSON to search for images, then select references for generation.

Search Targets
People / settings

→

Naver Search
Candidates for each article

→

Claude Review
Verify and select candidates

→

Generation Input
Person / background references

```
// Run image search for each article
spawn("node", [naverSearchPath, tempFile], {
  stdio: "inherit",
  env: { ...process.env, TEMP_INPUT_FILE: tempFile }
});

// Select images after all searches complete
spawn("node", [imageSelectorPath], {
  stdio: "inherit"
});
```

1. Save each row as a temporary JSON file
2. Pass its path to naver_search.js
3. Run image_selector.js after all searches succeed

Selected reference image paths

image_selected/row_{n}/slot_{i}/

protagonist1.jpg / protagonist2.jpg

background.jpg

Source: run_naver.js, create_image.py. Claude review reflects the operating workflow; detailed criteria reside in image_selector.js.

Animate selected scenes from generated images

Image and video generation are handled separately for each slot.

Gemini Image Generation

Use Claude-selected person and background references

gemini-2.5-flash-image / 9:16 output

Up to three attempts with increasing waits on API errors

Save generated.jpg in article and slot directories

Veo Video Generation

Process only slots where media_type is video

google/veo-3.1-fast / 4 sec / 720p / 9:16

generate_audio: false keeps narration separate

Skip regeneration when generated.mp4 already exists

create_video.js

```
if (slot.media_type !== 'video') continue;
try {
  await fs.access(videoPath);
  totalSuccess++;
  continue;
} catch {}
```

Skip completed videos on reruns

Check the media type and existing video file.

Skip completed slots to avoid duplicate API calls.

A separate rule also skips slot 4 for FACT_FOCUS.

Source: run_naver.js, create_image.py, create_video.js. Image search and selection run as child processes.

Align subtitles and edits with generated speech

Script slot boundaries are preserved in the input to speech generation.

Voice Processing in Code

Prepare slot text as JSON input for TTS

TTS_NUM selects a worker from 1 to 6

Run tts_sync.js after speech generation succeeds

Stop subsequent steps if a child process fails

Workflow Used in Operation

Generate narration with Naver TTS

Extract timestamps from audio with Whisper

Adjust subtitle timing for each sentence

Automate FFmpeg editing and channel uploads

run_tts.js

```
await runCommand(
  "node", [ttsWorkerFile], ttsStyleDir
);
await runCommand(
  "node", ["tts_sync.js"], utilsDir
);
```

Generate speech, then synchronize

runCommand resolves when the exit code is 0.

A rejection stops execution before the next await.

Re-analyze generated audio to refine subtitle timing.

Source: prepare_tts.js, run_tts.js. Tools used: Naver TTS, Whisper and FFmpeg.

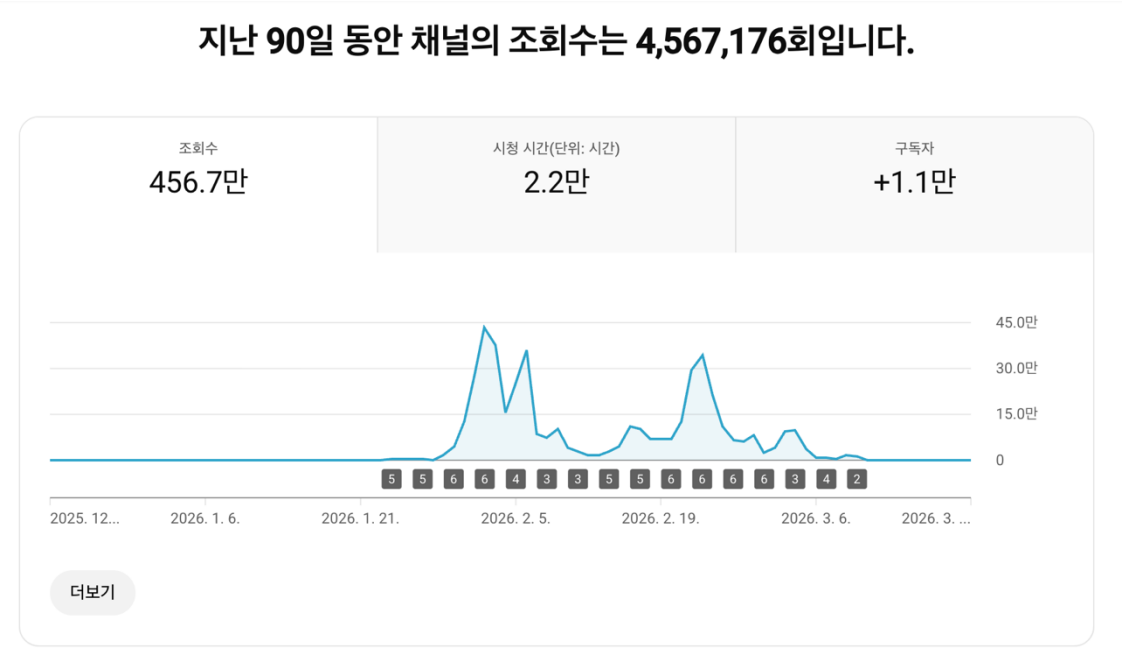
Separate model decisions from processing rules

The model evaluates content while code handles deterministic processing.

Scoring	GPT evaluates individual headline criteria	Code sums, sorts and sets the selection count
Data Linking	Parse GPT output as JSON	Keep the original row_number to preserve article identity
Reruns	Retry failed image generation	Skip existing video files to avoid regeneration
Execution Order	Run the TTS worker as a child process	Proceed to synchronization only on exit code 0

Channel Results

Results captured in the channel's analytics dashboard.



90 days: 4,567,176 views / 22K watch hours / +11K subscribers

Connecting generative AI to a repeatable workflow

Development experience from this personal project and areas for improvement.

What I Built

News collection with Google Sheets integration
GPT scoring, filtering and script-to-scene workflows
Media generation through Gemini and Veo APIs
Operation of the voice and editing pipeline

Next Improvements

Stronger validation of score ranges
Isolate failed tasks and manage reruns

Experience relevant to AI service development

I connected API outputs through shared data formats and organized stages around failures and reruns. Operating the channel showed me the need to deliver model outputs in a form that downstream tasks can use.